

CORRECTIONS & CODE · DATA SCIENCE

Corrections

Le code complet des tâches du cahier du participant, thème par thème.

À lire d'abord : n'ouvrez une correction qu'après avoir vraiment essayé et épuisé les indices du cahier. Le but est de **comprendre** la solution, pas de la recopier. Ces extraits supposent les données déjà chargées par le starter kit.

Système de recommandation

Rappel des données : ratings (userId, movieId, rating) et movies (movieId, title, genres).

A.2 — Recommander par popularité

On regroupe par film, on calcule note moyenne ET nombre de notes, puis on filtre avant de trier.

```
stats = (ratings.groupby("movieId")
         .agg(note_moy=("rating", "mean"), nb=("rating", "count"))
         .reset_index())

# On ne garde que les films assez notes (>= 10) avant de trier
populaires = stats[stats["nb"] >= 10].sort_values("note_moy", ascending=False)
top5 = populaires.merge(movies, on="movieId").head(5)
print(top5[["title", "note_moy", "nb"]])
```

A.3 — Ceux qui ont aimé X ont aussi aimé...

On trouve les fans du film, puis les autres films qu'ils ont bien notés.

```
def similaires(movie_id, n=5):
    # 1. utilisateurs ayant bien noté ce film
    fans = ratings[(ratings["movieId"] == movie_id) &
                  (ratings["rating"] >= 4)][["userId"]]
    # 2. autres films bien notés par ces memes utilisateurs
    autres = ratings[(ratings["userId"].isin(fans)) &
                    (ratings["movieId"] != movie_id) &
                    (ratings["rating"] >= 4)]
    reco = autres.groupby("movieId").size().sort_values(ascending=False).head(n)
    return movies[movies["movieId"].isin(reco.index)][["title"]]

print(similaires(1))
```

A.5 — Filtre genre + démarrage à froid

On filtre les films par genre, puis on applique la reco par popularité à ce sous-ensemble.

```
def reco_genre(genre, n=5):
    ids = movies[movies["genres"].str.contains(genre)][["movieId"]]
    s = stats[(stats["movieId"].isin(ids)) & (stats["nb"] >= 5)]
    s = s.sort_values("note_moy", ascending=False).head(n)
    return movies[movies["movieId"].isin(s["movieId"])]["title"]

# Nouvel utilisateur : on ne connaît que son genre préféré
print(reco_genre("Comedy"))
```

A.6 — Exclure les films déjà vus

On retire du resultat les films déjà notes par l'utilisateur.

```
def deja_vus(user_id):
    return set(ratings[ratings["userId"] == user_id]["movieId"])

def reco_hors_vus(user_id, candidats, n=5):
    vus = deja_vus(user_id)
    return [m for m in candidats if m not in vus][:n]
```

A.7 — Similarite par cosinus (item-based)

Matrice utilisateurs x films, puis similarite cosinus entre films.

```
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

mat = ratings.pivot_table(index="userId", columns="movieId",
                           values="rating").fillna(0)
sim = pd.DataFrame(cosine_similarity(mat.T),
                   index=mat.columns, columns=mat.columns)

def reco_item(movie_id, n=5):
    proches = sim[movie_id].sort_values(ascending=False).iloc[1:n+1].index
    return movies[movies["movieId"].isin(proches)][["title"]]

print(reco_item(1))
```

A.8 — Evaluer (approche simple)

On cache une note connue et on regarde si le film revient dans le top recommande.

```
def test_utilisateur(user_id, n=10):
    aimes = ratings[(ratings["userId"] == user_id) &
                    (ratings["rating"] >= 4)]["movieId"].tolist()
    if not aimes:
        return None
    cible = aimes[0]
    recos = reco_item(cible, n=n)
    # ici : inspection qualitative, ou compter les recoupements
    return cible, recos

# Fais tourner sur 5 utilisateurs et compte les bons recoupements.
```

Détection d'anomalies

Rappel des données : df avec une colonne amount (montant), t (ordre) et, si disponible, is_fraud (0/1).

B.2 — Detection par z-score

z = écart à la moyenne en nombre d'écarts-types ; anomalie si $|z| > 3$.

```
df["z"] = (df["amount"] - df["amount"].mean()) / df["amount"].std()
df["anomalie_z"] = df["z"].abs() > 3
print("Cas signales (z-score) :", df["anomalie_z"].sum())
```

B.3 — Detection par IQR (quartiles)

On borne avec $Q1 - 1.5 * IQR$ et $Q3 + 1.5 * IQR$.

```
q1, q3 = df["amount"].quantile([0.25, 0.75])
iqr = q3 - q1
borne_bas, borne_haut = q1 - 1.5*iqr, q3 + 1.5*iqr
df["anomalie_iqr"] = (df["amount"] < borne_bas) | (df["amount"] > borne_haut)
print("Cas signales (IQR) :", df["anomalie_iqr"].sum())
```

B.4 — Visualiser les anomalies

Deux nuages superposés : normaux, puis anormaux en rouge.

```
import matplotlib.pyplot as plt

normaux = df[~df["anomalie_iqr"]]
anormaux = df[df["anomalie_iqr"]]
plt.scatter(normaux["t"], normaux["amount"], s=8, label="Normal")
plt.scatter(anormaux["t"], anormaux["amount"], s=25, color="red", label="Anomalie")
plt.legend(); plt.xlabel("Ordre"); plt.ylabel("Montant"); plt.show()
```

B.5 — Faire varier le seuil

On observe l'effet du seuil sur le nombre d'alertes.

```
for seuil in [2.0, 2.5, 3.0, 3.5]:
    n = (df["z"].abs() > seuil).sum()
    print(f"Seuil {seuil} -> {n} cas signales")
```

B.6 — Mesurer avec les étiquettes

Matrice de confusion entre alertes et vraies fraudes.

```
from sklearn.metrics import confusion_matrix

tn, fp, fn, tp = confusion_matrix(df["is_fraud"],
                                  df["anomalie_iqr"].astype(int)).ravel()
print(f"Vraies fraudes attrapées : {tp}")
print(f"Fraudes ratees           : {fn}")
print(f"Fausses alertes          : {fp}")
```

B.7 — Isolation Forest & comparaison

Un modele dedie, puis meme matrice de confusion pour comparer au socle.

```
from sklearn.ensemble import IsolationForest
from sklearn.metrics import confusion_matrix

iso = IsolationForest(contamination=0.05, random_state=0)
df["anomalie_iso"] = (iso.fit_predict(df[["amount"]]) == -1).astype(int)

tn, fp, fn, tp = confusion_matrix(df["is_fraud"], df["anomalie_iso"]).ravel()
print(f"IsolationForest -> attrapees {tp}, ratees {fn}, fausses {fp}")
# Compare ces chiffres a ceux de B.6 : le modele fait-il mieux ?
```

Data storytelling

Rappel des données : data avec par ex. region, annee, population, depense. Adapte les noms de colonnes à ton jeu de données.

C.1 — Nettoyer les donnees

Reperer les trous, retirer doublons et lignes inexploitablees.

```
print(data.isna().sum()) # ou sont les valeurs manquantes ?
data = data.dropna(subset=["depense"]) # ou remplir : .fillna(data["depense"].median())
data = data.drop_duplicates()
```

C.2 — Trois visualisations lisibles

Un graphique = une idee. Titre clair, axes legendes.

```
import matplotlib.pyplot as plt

# 1. Evolution dans le temps -> courbe
data.groupby("annee")["depense"].mean().plot(title="Depense moyenne par annee")
plt.ylabel("Depense moyenne"); plt.show()

# 2. Comparaison entre categories -> barres
data.groupby("region")["depense"].mean().plot(kind="bar",
        title="Depense moyenne par region")
plt.ylabel("Depense moyenne"); plt.show()

# 3. Distribution -> histogramme
data["depense"].plot(kind="hist", bins=20, title="Repartition des depenses")
plt.xlabel("Depense"); plt.show()
```

C.4 — Croiser deux dimensions

Tableau croise region x annee.

```
croise = data.pivot_table(index="annee", columns="region",
        values="depense", aggfunc="mean")
print(croise.round(1))
croise.plot(title="Depense moyenne par region et par annee") # une courbe par region
```

C.6 — Variable calculee

Une mesure relative (par habitant) est souvent plus juste que la valeur brute.

```
data["depense_par_hab"] = data["depense"] / data["population"]
# Compare le classement des regions AVANT et APRES cette transformation :
print(data.groupby("region")["depense"].mean().round(1))
print(data.groupby("region")["depense_par_hab"].mean().round(3))
```

C.7 — Tendance simple

Une droite de tendance sur la serie temporelle.

```
import numpy as np, matplotlib.pyplot as plt

serie = data.groupby("annee")["depense"].mean()
pente, ordo = np.polyfit(serie.index, serie.values, 1)
plt.plot(serie.index, serie.values, marker="o", label="Observe")
plt.plot(serie.index, pente*serie.index + ordo, "--", label="Tendance")
plt.legend(); plt.title(f"Tendance : {pente:+.1f} / an"); plt.show()
```

C.8 — DEFI : tableau de bord interactif

Deux voies au choix. SANS CODE : Looker Studio (studio.google.com) -> connecte ton fichier, ajoute des graphiques + un bloc statistique par indicateur, puis un controle de filtre lie a la region. AVEC CODE : le script Streamlit ci-dessous (fichier dashboard.py, lance avec la commande streamlit run dashboard.py ; s'exécute en local ou sur Streamlit Community Cloud, pas dans un notebook Colab).

```
# dashboard.py -> lancer avec : streamlit run dashboard.py
import streamlit as st
import pandas as pd

data = pd.read_csv("donnees_nettoyes.csv") # ton fichier apres nettoyage (C.1)

st.title("Tableau de bord des depenses")

# --- Filtre interactif ---
regions = ["Toutes"] + sorted(data["region"].unique())
choix = st.selectbox("Region", regions)
d = data if choix == "Toutes" else data[data["region"] == choix]

# --- Indicateurs cles (KPI) : se recalculent selon le filtre ---
c1, c2, c3 = st.columns(3)
c1.metric("Depense moyenne", f"{d['depense'].mean():.0f}")
c2.metric("Population totale", f"{int(d['population'].sum()):,}")
c3.metric("Nb de lignes", len(d))

# --- Graphiques ---
st.subheader("Evolution par annee")
st.line_chart(d.groupby("annee")["depense"].mean())

st.subheader("Comparaison entre regions")
st.bar_chart(data.groupby("region")["depense"].mean())
```